

SDKB v1.0 온톨로지 큐레이션 구축 실행계획

Executive Summary

본 보고서는 큐레이션 방식의 온톨로지 구축을 전제로, 첨부 파일 **semiconductor_v0_3(JSON 그래프)**를 베이스라인으로 삼아 **SDKB v1.0(반도체 도메인 지식 베이스)**를 12 주 이내에 “공개 가능한 Open Core”와 “유료/제한 소스 Link-Only”를 명확히 분리하여 구축하는 최적 실행계획을 제안합니다. 핵심은 (1) 베이스라인의 14 개 노드 타입/관계 스키마를 **OWL+JSON-LD 이중 스키마**로 고도화하고, (2) SemiKong·SemicONTO·SEMI(E10/E30/E40/E116)·NIST CSF 2.0/IR 8546·ECHA SCIP·BIS(EAR/Entity List)와의 **정합(ontology alignment)**을 자동+전문가 검토 혼합 방식으로 수행하며, (3) **PROV-O 기반 프로비넌스**와 **SHACL 기반 품질검증**을 릴리스 게이트로 고정하는 것입니다. PROV-O 는 상호운용 가능한 출처/활동/주체 표현을 위한 W3C 표준 온톨로지이며, SDKB 의 “큐레이션 기반 신뢰성”을 데이터 수준에서 보장하는 가장 강력한 선택지입니다. [1]

요청사항에 따라 작업은 **사용자 활성 커넥터(google_drive, Canva)**를 우선 사용해 semiconductor_v0_3 를 불러오려 시도했습니다(Drive 내 검색 결과 semiconductor_v0_3 동일 파일은 확인되지 않았고, Canva 검색은 관련 디자인 자산을 찾지 못했습니다). 따라서 분석은 첨부된 /mnt/data/semiconductor_v0_3.json 을 직접 파싱해 구조(클래스·속성·관계·메타데이터)를 자동 추출한 결과를 기반으로 수행했습니다.

필수 학습 항목(권장 6 개)

- 1) **베이스라인 파일 구조/노드타입/관계 타입(스키마) 파악 및 제약조건 정의**(JSON 스키마 + OWL/SHACL 로 이중화)
- 2) **라이선스·저작권 리스크 분류와 Open Core/Link-Only 분리 규칙**(특히 SEMI/JEDEC/IRDS 등 유료·접근제한 소스) — SEMI 표준은 유료 PDF 로 판매되고 라이선스 조건이 동반됩니다. [2]
- 3) **규제 임계값·조항 모델링**(BIS EAR/Entity List 및 15 CFR 744.23 등) — Entity List 와 반도체 제조장비(SME) 관련 end-use 통제가 명시돼 있어 규칙 노드화가 필요합니다. [3]
- 4) **FMEA 인과모델 요구사항**(FBSFM 통합, 신뢰도/증거 표현) — FBSFM 은 설계 지식과

FMEA 를 온톨로지로 통합하는 프레임워크로 SDKB 의 인과 그래프 고도화에 직접적 근거를 제공합니다. [4]

5) **SHACL/PROV-O/JSON-LD 매핑 규칙**(필수 속성, 범위/형식 검증, 프로비넌스 추적 표준화) — SHACL 은 RDF 그래프 검증 표준이며 JSON-LD 는 링크드데이터 직렬화 표준입니다. [5]

6) **릴리스 패키징/Hugging Face 데이터셋 카드/라이선스 표기**(CDLA-Permissive-2.0 적용, CC-BY 등 혼합 라이선스 정리) — Dataset Card 의 메타데이터/문서화와 CDLA-Permissive-2.0 호환성 가이드를 준수해야 합니다. [6]

Baseline semiconductor_v0_3 구조 자동 추출 분석

메타데이터 및 구성요소(자동 추출 결과)

semiconductor_v0_3.json 은 상위 메타데이터와 그래프 본문으로 구성됩니다.

- 상위 키: version_id, version, status, created_by, notes
- 그래프 본문: nodes(198), edges(264)
- 보조 테이블: synonyms(86), cases(3)
- 출처 레지스트리: provenance_sources(다수 소스의 full_name/license/url/format/note)
- 품질 통계: provenance_statistics(검증 필요 노드/엣지 수, 해석 유형 분포 등)

특히 provenance_statistics 에 **검증 필요(validation_required) 노드 71 개, 엣지 117 개**가 별도 집계되어 있어, SDKB v1.0 에서는 이를 **전문가 리뷰 큐 자동 생성의 1 차 기준**으로 활용할 수 있습니다(후술).

14 개 노드 타입(클래스) 현황

베이스라인은 이미 **14 개 타입**을 사용하고 있어, 사용자 계획(“14 개 노드 타입 기반 큐레이션”)과 구조적으로 일치합니다.

검증 필요 플래그 분포(자동 추출)			
노드 타입	개수	특징(요약)	추출
Process	8	공정 대분류	0

노드 타입	개수	특징(요약)	검증 필요 플래그 분포(자동 추출)
SubProcess	12	공정 세부 단계	4
EquipmentClass	11	장비 클래스	0
Equipment	42	장비 인스턴스(모델/벤더 포함)	15
Vendor	15	공급업체	0
Organization	2	조직(컨소시엄/기관 등)	0
Parameter	5	공정 파라미터(단위 예시 포함)	0
Metrology	3	계측/검사 방법	0
Material	20	소재 카테고리	0
TechnologyNode	3	기술노드(예: 7nm 등)	0
FailureMode	25	불량/고장 모드	0
RootCause	20	원인	20
Mitigation	20	완화/대응	20
Skill	12	필요 역량	12

시사점: SDKB v1.0 설계에서 “타입 추가(확장)”는 가능하지만, **호환성을 최우선**으로 하면 현재 14 개 타입을 **Core 타입으로 고정**하고, 규제/표준/출처를 위한 추가 타입은 **Governance 레이어에서만 확장**하는 방식이 안정적입니다.

관계(엣지) 타입 및 가중치

엣지는 기본적으로 {src, predicate, dst, weight, provenance} 구조이며 일부는 description, context_modifiers 를 가집니다.

predicate(관계)	개수	의미(권장 OWL 속성명)	검증 필요 플래그
IS_INSTANCE_OF	42	equipment → equipmentClass	0
PROVIDED_BY	42	equipment → vendor	0

predicate(관계)	개수	의미(권장 OWL 속성명)	검증 필요 플래그
CAUSED_BY	36	failureMode → rootCause	36
USES_MATERIAL	30	subprocess → material	0
OBSERVED_IN	27	failureMode → subprocess	27
MITIGATED_BY	21	failureMode → mitigation	21
REQUIRES_SKILL	18	subprocess/mitigation → skill	18
HAS_SUBPROCESS	12	process → subprocess	0
USES_EQUIPMENT_CLASS	10	subprocess → equipmentClass	0
INCOMPATIBLE_WITH	8	subprocess → material	8
NOT_ALLOWED_WITH	7	subprocess → material	7
MADE_BY	5	material → vendor	0
HAS_PARAMETER	4	subprocess → parameter	0
MEASURED_BY	2	subprocess → metrology	0

가중치(weight)는 일부 엣지에 **-1.0 ~ 1.0** 범위로 존재하며, 음수는 “금지/비호환” 성격의 제약을 나타내는 용도로 쓰이고 있습니다. SDKB v1.0 에서는 이를 단순 수치로 두기보다, **제약(constraint) 타입을 명시적으로 분리**(예: sdkb:hardConstraint, sdkb:softConstraint)하고, weight 는 **확률/신뢰도(confidence)**로 의미를 고정하는 것이 재현성과 해석 가능성 측면에서 유리합니다.

SDKB v1.0 목표 아키텍처

레이어링 원칙: Domain / Governance / Link-Only

SDKB v1.0 의 가장 중요한 설계 결정을 **레이어 분리**로 제안합니다.

- **Domain(Open Core):**
공정·장비·소재·파라미터·계측·기술노드·FMEA(불량/원인/완화/스킬) 등 “반도체 기술경영 의사결정”에 직접 필요한 **사실/개념 그래프**
- **Governance(Open Core):** 규제/보안/컴플라이언스(예: NIST CSF 2.0·IR 8546, BIS EAR, ECHA SCIP) 지식을 **규칙/임계값/적용 범위**로 정형화
- NIST CSF 2.0 는 2024-02-26 에 공개된 최신 프레임워크이며, “Govern” 기능 추가와 공급망 강조가 명시됩니다. [\[7\]](#)

- NIST IR 8546(초안)는 CSF 2.0 기반 반도체 제조 프로파일로, 산업 생태계의 복잡성과 위험기반 접근을 “원하는 보안 성과(outcomes)” 관점에서 제시합니다. [8]
- **Link-Only:** 유료/제한/재배포 불명확 소스(예: SEMI 표준 PDF, JEDEC 문서, IRDS 로드맵 등)는 **문서 링크/식별자/절(section) 메타데이터**만 보유하고 본문·표·정의문을 SDKB 에 포함하지 않음 — SEMI E10/E30/E40/E116 은 스토어에서 유료 판매되며 표준 저작권/라이선스 정책이 동반됩니다. [9]

아래는 레이어 구조(권장)입니다.

```
graph TD
    subgraph OpenCore_Domain[Open Core: Domain Layer]
        P[Process/SubProcess]
        EQC[EquipmentClass]
        EQ[Equipment]
        M[Material]
        PAR[Parameter]
        MET[Metrology]
        TN[TechnologyNode]
        FMEA[FailureMode/RootCause/Mitigation/Skill]
    end

    subgraph OpenCore_Gov[Open Core: Governance Layer]
        NIST[NIST CSF 2.0 / IR 8546 Profile]
        BIS[BIS EAR / Entity List / 744.23]
        ECHA[ECHA SCIP rules]
        RULES[Rule/Threshold/Applicability nodes]
    end

    subgraph LinkOnly[Link-Only Layer (Not redistributed)]
        SEMI[SEMI E10/E30/E40/E116 references]
        JEDEC[JEDEC references]
        IRDS[IRDS references]
    end

    P --> EQC
    P --> M
    P --> PAR
    P --> MET
    P --> FMEA
    TN --> P

    OpenCore_Domain --> RULES
    RULES --> NIST
    RULES --> BIS
```

RULES --> ECHA

RULES --> SEMI

RULES --> JEDEC

RULES --> IRDS

공개 라이선스 전략

- SDKB v1.0 **Open Core 데이터**는 **CDLA-Permissive-2.0** 적용을 기본으로 하되, 혼합 출처(예: CC-BY-4.0 데이터)의 경우 **저작자 표시/출처 표기 유지**가 필요합니다.
CDLA-Permissive-2.0 의 SPDX 식별자와 텍스트는 SPDX 및 CDLA 공식 자료에서 제공됩니다. [10]
- “Link-Only”는 **재배포 대상에서 제외**하거나, 포함하더라도 **식별자/URL/문서명/개정본** 수준으로 제한합니다(본문·표·정의문 미포함).

12 주 실행 파이프라인과 자동화

주차별 실행계획(12 주 이내 권장)

주차	핵심 목표	주요 작업(요약)	산출물	책임 역할(주관/지원)
1 주	베이스라인 완전 파악	semiconductor_v0_3 파싱, 스키마/통계 자동 추출, 결측/중복/참조 무결성 점검	구조리포트, JSON Schema v0.3, 노드/엣지 요약표	데이터 엔지니어(주), 온톨로지 엔지니어
2 주	SDKB v1.0 메타모델 확정	OWL 클래스/속성 설계, ID/네임스페이스 정책, Open Core/Link-Only 분리 규칙 확정	sdkb-core.owl 초안, 네임스페이스/ID 가이드	온톨로지 엔지니어(주), PM/리드
3 주	변환 파이프라인 구축	JSON→RDF(TTL), JSON→JSON-LD, Parquet 추출 스크립트 구축	core.ttl, core.jsonld, nodes.parquet, edges.parquet(초안)	데이터 엔지니어(주)
4 주	정합 엔진	lexical+embedding	mappings_candidates.t	온톨로지

				책임 역할(주관/지원)
주차	핵심 목표	주요 작업(요약)	산출물	
	v1	하이드브리드 매핑 파이프라인, 후보 매핑 TSV 생성, 리뷰 UI/워크플로우 설계	sv, 리뷰 규칙서	엔지니어(주), 데이터 엔지니어
5 주	SemiKong 정합	SemiKong 공정 온톨로지(부록 표 기반) Process/SubProcess 계층 보강 및 매핑 확정	semikong_alignment.ts v, 계층 보강본	온톨로지 엔지니어(주), 반도체 공정 SME
6 주	SemicONTO 정합	SemicONTO(OWL/Turtle) 와 클래스/속성 정합, SKOS 매핑(정밀/근접) 확정	semicononto_alignment.t t1, 매핑 QA 리포트	온톨로지 엔지니어(주)
7 주	SEMI Link- Only 구축	E10/E30/E40/E116 표준 메타데이터/참조 그래프 구성(본문 미포함), 장비 상태/통신/가공관리 개념 매핑	semi_links.ttl, 표준 참조 카탈로그	규제/표준 SME(주), 온톨로지 엔지니어
8 주	NIST Governanc e 구축	CSF 2.0 기능/성과(outcomes) 노드화, IR 8546 프로파일과 연결	nist_gov.ttl, 질의 예제(보안/공급망)	보안 SME(주), 온톨로지 엔지니어
9 주	BIS Governanc e 구축	Entity List/744.23/CCL 관련 규칙.적용범위 모델링, 임계값.조건(국가군/용도) 코드화	bis_gov.ttl, 규제 질의 샘플	규제 SME(주), 데이터 엔지니어
10	ECHA SCIP Governanc	SCIP 신고 조건(0.1%	echa_gov.ttl, 소재-규제	규제 SME(주)

주차	핵심 목표	주요 작업(요약)	산출물	책임 역할(주관/지원)
주	e 구축	w/w, 2021-01-05~) 규칙화, 소재 노드와 연결	연결 QA	
11 주	검증/QA 게이트	SHACL shapes 완성, 유닛/통합 테스트, 전문가 리뷰(Validation- required 중심)	shapes.ttl, 테스트 리포트, 리뷰 승인 로그	QA(주), SME, 온톨로지 엔지니어
12 주	릴리스	HF 패키징(OWL/TTL, JSON-LD, Parquet, 카드), 버전 태깅/변경이력	SDKB v1.0 릴리스 번들, Dataset Card, SPARQL 예제	PM(주), 데이터 엔지니어

파이프라인 타임라인(mermaid)

gantt

dateFormat YYYY-MM-DD

title SDKB v1.0 (12 주) 파이프라인

section Foundation

Baseline 파싱/스키마 추출 :a1, 2026-04-13, 7d

메타모델(OWL/ID/레이어) 확정 :a2, after a1, 7d

변환 파이프라인(JSON→RDF/JSON-LD) :a3, after a2, 7d

section Alignment

매핑 엔진(lexical+embedding) :b1, after a3, 7d

SemiKong 정합 :b2, after b1, 7d

SemicONTO 정합 :b3, after b2, 7d

SEMI Link-Only 참조 그래프 :b4, after b3, 7d

section Governance

NIST CSF2.0/IR8546 모델링 :c1, after b4, 7d

BIS(EAR/EntityList/744.23) 규칙화 :c2, after c1, 7d

ECHA SCIP 규칙화 :c3, after c2, 7d

section QA & Release

SHACL/테스트/전문가 리뷰 :d1, after c3, 7d

릴리스 패키징/카드/예제 질의 :d2, after d1, 7d

단계별 산출물·도구·조직(요약 표)

단계	구체 산출물	권장 도구/기술	자동화 포인트	주 책임
수집(Ingest)	원천 JSON 고정본, 파일 해시, 스키마 리포트	Python, Pandas	해시/무결성 검사, 참조 무결성 검사	데이터 엔지니어
정규화(Normalize)	정규화 규칙서, 표준 라벨/동의어 사전	Python, RapidFuzz	라벨 정규화, 동의어 병합, ID slug 규칙	온톨로지 엔지니어
정합(Align)	매핑 후보 TSV, 확정 매핑 TTL/TSV	임베딩 모델, LogMap/AML	후보 생성/스코어링/필터링	온톨로지 엔지니어
프로비넌스 부착	PROV-O 그래프/필드 매핑, license 필드	PROV-O, JSON-LD	prov:Activity 자동 생성, 변경이력 기록	데이터 엔지니어
검증(Validate)	SHACL shapes, 테스트 리포트	SHACL validator	CI 에서 SHACL/유닛 테스트 강제	QA/온톨로지 엔지니어
릴리스(Release)	OWL/TTL, JSON-LD, Parquet, 카드	Parquet, TTL, HF 카드	릴리스 번들 자동 생성	PM/데이터 엔지니어

LogMap 은 대규모 온톨로지 매칭에서 논리 기반 진단/수정까지 지원하는 계열 도구로 보고되며, 대형 정합에서 “논리적 불일치”를 줄이는 데 유용합니다. [11]

AgreementMakerLight(AML)은 대형 온톨로지 매칭을 효율적으로 수행하도록 설계된 시스템으로 보고되어, SDKB 처럼 레퍼런스 소스가 늘어나는 상황에서 실용적인 선택지입니다. [12]

자동화 스크립트 예시

스키마/통계 자동 추출 (입력: semiconductor_v0_3.json → 출력: schema_report.json, nodes.parquet, edges.parquet)

```
# input : semiconductor_v0_3.json
# output: schema_report.json, nodes.parquet, edges.parquet
```

```
import json
import pandas as pd
from collections import Counter
```

```
IN_PATH = "semiconductor_v0_3.json"
OUT_SCHEMA = "schema_report.json"
OUT_NODES = "nodes.parquet"
OUT_EDGES = "edges.parquet"
```

```
with open(IN_PATH, "r", encoding="utf-8") as f:
    g = json.load(f)
```

```
nodes = g["nodes"]
edges = g["edges"]
```

```
node_types = Counter(n["type"] for n in nodes)
pred_types = Counter(e["predicate"] for e in edges)
```

```
node_df = pd.DataFrame(nodes)
edge_df = pd.DataFrame(edges)
```

```
schema_report = {
    "version": g.get("version"),
    "version_id": g.get("version_id"),
    "counts": {"nodes": len(nodes), "edges": len(edges)},
    "node_types": dict(node_types),
    "predicates": dict(pred_types),
    "has_synonyms": "synonyms" in g,
    "has_cases": "cases" in g,
    "provenance_sources": list(g.get("provenance_sources", {}).keys()),
    "provenance_statistics": g.get("provenance_statistics", {}),
}
```

```
}
```

```
with open(OUT_SCHEMA, "w", encoding="utf-8") as f:  
    json.dump(schema_report, f, ensure_ascii=False, indent=2)
```

```
node_df.to_parquet(OUT_NODES, index=False)  
edge_df.to_parquet(OUT_EDGES, index=False)  
print("done")
```

JSON→RDF(Turtle) 변환 (입력: JSON → 출력: sdkb_core.ttl)

```
# input : semiconductor_v0_3.json  
# output: sdkb_core.ttl
```

```
import json  
from rdflib import Graph, Namespace, URIRef, Literal  
from rdflib.namespace import RDF, RDFS, DCTERMS, SKOS
```

```
BASE = "https://example.org/sdkb/"  
SDKB = Namespace(BASE)  
PROV = Namespace("http://www.w3.org/ns/prov#")
```

```
def uri_from_id(node_id: str) -> URIRef:  
    return URIRef(BASE + node_id.replace(":", "/"))
```

```
with open("semiconductor_v0_3.json", "r", encoding="utf-8") as f:  
    g = json.load(f)
```

```
rdf = Graph()  
rdf.bind("sdkb", SDKB)  
rdf.bind("skos", SKOS)  
rdf.bind("dcterm", DCTERMS)  
rdf.bind("prov", PROV)
```

```
# nodes  
for n in g["nodes"]:  
    u = uri_from_id(n["id"])  
    rdf.add((u, RDF.type, SDKB[n["type"]]))  
    rdf.add((u, SKOS.prefLabel, Literal(n["canonical_name"], lang="en")))  
    if n.get("description"):  
        rdf.add((u, SKOS.definition, Literal(n["description"], lang="en")))  
  
    prov = n.get("provenance", {})  
    if prov.get("source"):  
        rdf.add((u, DCTERMS.source, Literal(prov["source"])))  
    if prov.get("license"):  
        rdf.add((u, DCTERMS.license, Literal(prov["license"])))  
    if prov.get("reference"):
```

```

        rdf.add((u, DCTERMS.bibliographicCitation, Literal(prov["reference"]))
    ))

# synonyms → skos:altLabel (언어 태그 유지)
for s in g.get("synonyms", []):
    u = uri_from_id(s["node_id"])
    rdf.add((u, SKOS.altLabel, Literal(s["term"], lang=s.get("lang", "und"))))
)

# edges
PRED = {
    "HAS_SUBPROCESS": SDKB.hasSubprocess,
    "USES_MATERIAL": SDKB.usesMaterial,
    "CAUSED_BY": SDKB.causedBy,
    "MITIGATED_BY": SDKB.mitigatedBy,
    "REQUIRES_SKILL": SDKB.requiresSkill,
    "PROVIDED_BY": SDKB.providedBy,
    "IS_INSTANCE_OF": SDKB.isInstanceOf,
}

for e in g["edges"]:
    s = uri_from_id(e["src"])
    o = uri_from_id(e["dst"])
    p = PRED.get(e["predicate"], SDKB[e["predicate"]])
    rdf.add((s, p, o))

rdf.serialize("sdkb_core.ttl", format="turtle")
print("done")

```

정합 후보 생성(lexical + embedding 하이브리드) (입력: labels → 출력: mapping_candidates.tsv)

input : sdkb_labels.csv, ref_labels.csv

output: mapping_candidates.tsv

note: embedding 모델은 예시이며 프로젝트 환경에 맞게 교체

```

import pandas as pd
from rapidfuzz import fuzz
from sentence_transformers import SentenceTransformer
import numpy as np

sdkb = pd.read_csv("sdkb_labels.csv") # columns: id, type, label
ref = pd.read_csv("ref_labels.csv") # columns: ref_id, ref_type, label

model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

sdkb_emb = model.encode(sdkb["label"].tolist(), normalize_embeddings=True)

```

```

ref_emb = model.encode(ref["label"].tolist(), normalize_embeddings=True)

rows = []
TOPK = 10

for i, (sid, slabel, stype) in enumerate(zip(sdkb["id"], sdkb["label"], sdkb["type"])):
    # cosine similarity top-k
    sims = ref_emb @ sdkb_emb[i]
    top_idx = np.argsort(-sims)[:TOPK]
    for j in top_idx:
        rlabel = ref.loc[j, "label"]
        lexical = fuzz.token_set_ratio(slabel.lower(), rlabel.lower()) / 100.
        0
        emb = float(sims[j])
        hybrid = 0.6 * emb + 0.4 * lexical
        rows.append([sid, stype, ref.loc[j, "ref_id"], ref.loc[j, "ref_type"],
                    lexical, emb, hybrid])

out = pd.DataFrame(rows, columns=["sdkb_id", "sdkb_type", "ref_id", "ref_type",
                                "lexical", "embedding", "hybrid"])
out.sort_values(["sdkb_id", "hybrid"], ascending=[True, False]).to_csv("mapping_candidates.tsv", sep="\t", index=False)

```

Ontology Alignment 전략과 소스별 정합 절차

매핑 알고리즘(lexical + embedding hybrid) 권장안

SDKB v1.0 정합은 후보 생성(자동) → 필터링(규칙/논리) → 전문가 확정(인간)의 3 단계로 설계하는 것이 가장 안정적입니다.

1) **Lexical 매칭**: canonical_name + synonyms 기반으로 토큰 정규화/약어

확장/편집거리(예: token_set_ratio)

2) **Embedding 매칭**: 라벨+정의문을 문장 임베딩으로 변환하여 코사인 유사도로 후보 상위 k 개 도출

3) **제약기반 필터링**:

- 타입 호환 매트릭스(예: Process↔ProcessStep, Equipment↔Equipment)
- 계층 일관성(상위-하위 매핑의 방향성 유지)

- 논리적 불일치 검출(대형 정합은 LogMap 류 진단/수정 접근이 유리) [11]

4) **전문가 확정**: validation_required=True 또는 hybrid score 경계 영역(예: 0.72~0.80)을 우선 검토

매핑 우선순위 규칙(라이선스·신뢰도·갱신빈도)

아래 매트릭스를 “우선 정합 대상 + 통합 방식” 결정 규칙으로 사용합니다.

소스	신뢰도(권위)	갱신/변경 빈도	라이선스 성격	권장 통합 방식	근거/주의
사용자 베이스라인(semiconductor_v0_3)	최우선(내부 기준)	프로젝트 관리	프로젝트 라이선스	Open Core	SDKB 의 기준 스키마/ID 원천
SemiKong	높은 편(원문+코드 공개)	중간	코드/가중치 Apache 2.0(깃허브 명시) [13] / 논문은 arXiv 에 서 라이선스 링크 제공 [14]	Open Core(공정 계층)	부록 표에 공정 계층 구조(공정群/모듈/유닛) 존재 [15]
SemicONTO	학술(온톨로지 엔지니어링)	중간	CEUR 워크숍 볼륨 CC BY 4.0 [16]	Open Core(스키마 정합)	실험/장비/소재 등 실험데이터 의미론 목표 [17]
SEMI(E10/E30/E40/E	산업	중간~높	유료	Link-Only	표준

소스	신뢰도(권위)	갱신/변경 빈도	라이선스 성격	권장 통합 방식	근거/주의
116)	표준(매우 높음)	음(개정 지속)	PDF/라이선스 정책(재 배포 제한 가능) [9]		메타데이터/개정본/식별자만
NIST CSF 2.0	정부 표준(매우 높음)	중간	공개 출판물 [18]	Open Core(Governance)	Govern 기능 추가/공급망 강조 [19]
NIST IR 8546 (IPD)	반도체 제조 프로파일(높음)	중간	공개 초안(IPD) [20]	Open Core(Governance)	반도체 제조 생태계/원하는 보안성과 중심 [8]
ECHA SCIP	EU 규정 기반(높음)	후보물질 업데이트 영향(높음)	규정/포털 정보는 공개, 데이터는 준수 필요	Open Core(규칙만)	0.1% w/w, 2021-01-05 부터 신고 의무 [21]
BIS(EAR/Entity List/744.23)	美 수출통제(매우 높음)	높음(규정 개정)	규정 텍스트는 공공 접근	Open Core(규칙/링크)	Entity List 개요 및 §744.23 SME 통제 조항 활용 [3]
MatKG	학술 데이터(중~높음)	낮음~중간	CC BY 4.0, 대규모 RDF/CS	Open Core(소재 확장)	소재 관련 엔티티/관계 확장에 유리 [23]

소스	신뢰도(권위)	갱신/변경 빈도	라이선스 성격	권장 통합 방식	근거/주의
			V 제공 [22]		
Wikidata	범용 식별자(중간)	높음(커뮤니티)	구조화 데이터 CC0 [24]	Open Core(식별자 링크)	외부 ID 정규화(owl:sameAs/스키마)

불일치 해결 정책(keep-source / merge / annotate)

- **keep-source**(기본값): 서로 다른 출처가 다른 관점의 분류를 제공하면, 두 개념을 유지하고 `skos:closeMatch/skos:exactMatch` 로 연결
- **merge**(강한 증거가 있을 때): 동일 개념으로 확증되면 SDKB 의 대표 노드로 병합하고, 원 ID 는 `sdkb:deprecatedId` 로 보존(검색 호환)
- **annotate**(충돌이 해소되지 않을 때): `sdkb:conflictNote`, `sdkb:reviewStatus="DISPUTED"`로 표시하고 릴리스에서 “검증 필요”로 유지

소스별 정합 절차(구체)

SemiKong 정합(공정 계층 확장)

SemiKong 논문은 공정 전반을 “Process Group / Process Module / Process Unit”로 정리한 상세 테이블(부록 A)을 제공합니다. [15] 이를 SDKB 의 Process/SubProcess 계층에 대응시키되, 다음 규칙을 권장합니다.

- Process Group → Process (예: Patterning)
- Process Module → SubProcess 상위 (예: Etching)
- Process Unit → SubProcess 하위(또는 SubProcess 에 `sdkb:granularity="unit"` 부여)
- 이미 베이스라인에 존재하는 노드는 **ID 유지**(호환성), SemiKong 항목은 **cross_ref** 로만 추가(merge 는 최소화)

또한 SemiKong 깃허브는 코드/가중치가 Apache 2.0 으로 배포된다고 명시합니다(베이스라인 계획의 “MIT” 언급과 달리, 실제 배포 라이선스는 Apache 2.0 으로 확인되므로 SDKB 의 라이선스 레지스트리를 이 기준으로 갱신해야 합니다). [13]

SemicONTO 정합(OWL 기반 스키마 정렬)

SemicONTO 는 반도체 실험데이터의 상호운용성 개선을 목표로 온톨로지를 제안합니다.

[17] SDKB v1.0 에서는 SemicONTO 를 “정합의 기준 스키마”로 쓰되, **owl:equivalentClass** 남용을 피하고 SKOS 매핑 중심으로 운영하는 것이 안정적입니다.

- Material ↔ (SemicONTO 의 Material 계열 클래스)
- Equipment/EquipmentClass ↔ (Equipment 계열)
- Process/SubProcess ↔ (ExperimentStep/ProcessStep 계열 후보)
- 실험 맥락(Experiment/Step/Result)이 SDKB 에 없으면 무리한 확장 대신 **Governance/Link-only** 로 분리하거나 v1.1 로드맵으로 이관

SemicONTO 는 GitHub 에서 공개 저장소로 운영되며, SPARQL 서버도 제공된다고 안내합니다. [25]

SEMI(E10/E30/E40/E116) 정합(산업 호환성 확보, 단 Link-Only 우선)

SEMI 표준은 구매형 PDF 로 제공되며 개정본이 지속 업데이트됩니다. [9] 따라서 SDKB 는 다음처럼 “링크.식별자 중심”으로 통합하는 것이 최선입니다.

- 표준별 **StandardReference** 노드(예: semi:E30-0725) 생성
- SDKB 의 Equipment/EquipmentClass/Parameter/Metrology 에 **sdkb:standardRef** 로 연결
- 표준의 상태모델/서비스 정의는 본문 반입 금지(필요시 표준 요약/공개 세미나 자료만 활용)

참고로 SEMI E10 스토어 페이지는 E10 이 **여섯 가지 상호배타적 장비 상태와**

RAM/Utilization 측정 방법을 정의한다고 밝힙니다. [26] GEM(E30)은 호스트-장비 통신 모델로 소개되며, SEMI 공식 튜토리얼에서도 E30 의 공식 명칭과 개념을 확인할 수 있습니다. [27]

NIST CSF 2.0 / IR 8546 정합(보안·공급망 거버넌스 레이어)

NIST CSF 2.0 는 사이버보안 성과(outcomes)의 분류 체계를 제공하며, 조직 규모/섹터에 무관하게 적용되도록 설계되었습니다. [18] 반도체 제조 프로파일(IR 8546 IPD)은 이를 반도체 제조 환경에 맞춘 커뮤니티 프로파일로서 “원하는 성과” 중심의 지침을 제공합니다. [8]

정합은 아래처럼 구현합니다.

- gov:NISTFunction(Govern/Identify/Protect/Detect/Respond/Recover) 노드화
- IR 8546 의 항목을 gov:Outcome 로 만들고 gov:belongsToFunction 연결
- SDKB 의 공급망/장비 통신/접근제어 관련 노드(Equipment, Vendor, Process)에 gov:relevantOutcome 로 링크

ECHA SCIP 정합(소재-규제 리스크)

SCIP 은 EU 시장에 공급되는 제품(articles) 중 Candidate List 의 SVHC 가 **0.1% w/w 초과** 포함된 경우, **2021-01-05 부터** ECHA 제출 의무가 있음을 명확히 합니다. [21]

SDKB 에서는 다음을 권장합니다.

- gov:SCIPReportingRule 노드(임계값 0.1% w/w, 적용일 2021-01-05)
- Material(또는 추후 Substance)에 gov:hasSVHC 및 gov:svhcConcentration 속성
- 질의 시 “소재→제품구성→SCIP 의무” 경로를 SPARQL 로 추출

BIS(EAR/Entity List/744.23) 정합(수출통제·거래 리스크)

BIS 의 Entity List 는 EAR Part 744 의 Supplement No.4 로 관리되며, 특정 거래 당사자가 Entity List 에 포함될 경우 CCL 품목 외에도 BIS 라이선스 요구가 추가될 수 있음을 명시합니다. [28] 또한 §744.23 은 “advanced-node ICs 및 semiconductor manufacturing equipment end use controls”를 규정하고, 특정 ECCN(예: 3B001 등)과 연계된 통제를 포함합니다. [29]

정합 구현은 다음을 권장합니다.

- gov:EARRule 노드: §744.16(Entity List), §744.23(SME/advanced-node ICs) 등
- gov:RegulatedItem 노드: ECCN 코드(예: 3B001 등)를 개체화
- SDKB EquipmentClass 에 gov:hasECCN(→RegulatedItem) 연결

- gov:applicability 에 국가군/목적지/최종사용(end-use)/지식(knowledge) 조건을 구조화

Provenance·FMEA·검증·배포 및 평가

PROV-O 기반 provenance·license 모델(필드 설계)

PROV-O 는 prov:Entity/Activity/Agent 를 통해 “어떤 데이터가 누구/무엇에 의해/어떤 과정을 거쳐 생성되었는지”를 표현하는 W3C 권고 표준입니다. [1] SDKB v1.0 에서는 베이스라인의 단일 provenance dict 를 **PROV-O 정규 모델로 승격**합니다.

권장 필드(노드/엣지 공통):

- prov:wasGeneratedBy → prov:Activity(예: “semikong_alignment_run_2026w18”)
- prov:wasDerivedFrom → 원천 문서/데이터셋(URI 또는 서지문헌)
- prov:wasAttributedTo → 에이전트(사람/팀/스크립트)
- dcterms:license → SPDX 식별자 또는 라이선스 URI
- sdkb:interpretationType → {verbatim | mapped | author-defined}
- sdkb:validationRequired → boolean
- sdkb:changeSet → 변경이력/릴리스 버전 태깅

PROV-O 스니펫 예시(개념)

```
@prefix sdkb: <https://example.org/sdkb/> .
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
```

```
sdkb:activity/align_semikong_2026w18
  a prov:Activity ;
  prov:used sdkb:source/semikong_paper ;
  prov:wasAssociatedWith sdkb:agent/ontology_engineer .
```

```
sdkb:process/lithography
  a sdkb:Process ;
  prov:wasGeneratedBy sdkb:activity/align_semikong_2026w18 ;
  dcterms:license "CDLA-Permissive-2.0" ;
  sdkb:validationRequired false .
```

JSON-LD 매핑 규칙(핵심)

JSON-LD 1.1 은 링크드데이터를 JSON 으로 직렬화하기 위한 W3C 권고 표준입니다. [30] SDKB v1.0 은 “RDF/OWL(TTL)”과 “JSON-LD”를 **동일 의미(동일 IRI)**로 제공해야 합니다.

JSON-LD context 스니펫(개념)

```
{
  "@context": {
    "sdbk": "https://example.org/sdbk/",
    "prov": "http://www.w3.org/ns/prov#",
    "dcterms": "http://purl.org/dc/terms/",
    "type": "@type",
    "id": "@id",
    "label": "http://www.w3.org/2000/01/rdf-schema#label",
    "license": "dcterms:license",
    "wasDerivedFrom": { "@id": "prov:wasDerivedFrom", "@type": "@id" }
  }
}
```

FMEA 인과관계 모델링: FBSFM 통합 방안

FBSFM(Function–Behavior–Structure–Failure Modes)은 시스템 설계 지식과 FMEA 를 온톨로지로 통합하기 위한 프레임워크로 소개되며, 실패모드-원인-영향의 다층 연결을 강조합니다. [4] SDKB v1.0 에서는 베이스라인의 FMEA 그래프를 유지하되, 아래 확장을 권장합니다.

1) 관계 타입(요청 반영)

- occursAtProcessStep : FailureMode → SubProcess (기존 OBSERVED_IN)
- isDueTo : FailureMode → RootCause (기존 CAUSED_BY)
- mitigatedBy : FailureMode → Mitigation (기존 MITIGATED_BY)
- 추가 권장:
- affectsMetric : FailureMode → Metric(예: LER, profile 등)
- requiresSkill : (SubProcess|Mitigation) → Skill (기존 REQUIRES_SKILL)

2) 신뢰도·증거 표현

- sdkb:confidence(0~1): 전문가/문헌 근거 기반 점수(베이스라인 weight 의미를 여기로 고정)
- sdkb:evidence(리스트):
 - source(문서/데이터셋), reference(서지), url, snippet_hash(본문 인용 대신 해시), reviewer, review_date
- sdkb:probability(선택): 원인→불량의 조건부 확률(데이터가 있을 때만)

3) **KG 기반 FMEA 활용 근거** FMEA 를 지식그래프로 확장하여 RAG/질의 성능을 향상시키려는 연구가 제안되며, 그래프 기반 의미질의의 장점이 보고됩니다. [31] SDKB 는 이를 “반도체 공정 FMEA 질의”에 적용할 수 있습니다.

SHACL 기반 검증·QA(예시 포함)

SHACL 은 RDF 그래프가 “형식/카디널리티/데이터타입/값 범위”를 만족하는지 검증하는 W3C 표준입니다. [32] SDKB v1.0 는 **릴리스 전 SHACL 검증 통과**를 필수 게이트로 설정합니다.

SHACL shapes 예시(필수 속성·값 범위)

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix sdkb: <https://example.org/sdkb/> .
@prefix dcterms: <http://purl.org/dc/terms/> .
```

```
sdkb:Shape_Node
  a sh:NodeShape ;
  sh:targetClass sdkb:Node ;
  sh:property [
    sh:path dcterms:license ;
    sh:minCount 1 ;
  ] ;
  sh:property [
    sh:path sdkb:interpretationType ;
    sh:in ("verbatim" "mapped" "author-defined") ;
    sh:minCount 1 ;
  ] .
```

```
sdkb:Shape_EdgeConfidence
  a sh:NodeShape ;
  sh:targetClass sdkb:Edge ;
  sh:property [
    sh:path sdkb:confidence ;
    sh:datatype sh:decimal ;
```

```

    sh:minInclusive 0.0 ;
    sh:maxInclusive 1.0 ;
  ] .

```

자동 테스트(유닛·통합) 권장

- 유닛 테스트(스키마):
 - ID 유일성, src/dst 참조 존재, 필수 필드 존재
- 통합 테스트(SPARQL 질의):
 - 핵심 질의 10 개(후술 3 개 포함) 결과 스냅샷 비교(회귀 테스트)
- 전문가 검토 워크플로우(검토대상 선정 규칙):
 - 1) validation_required=True 전체(우선)
 - 2) 중심성/빈도 기반 상위 10% 노드(추가)
 - 3) 규제/표준 연결 엣지 100% 샘플링(필수)

배포·릴리스 패키지 구성과 CDLA-Permissive-2.0 유의점

Hugging Face Dataset Card 는 README.md 가 렌더링되며, YAML 메타데이터로 라이선스/언어/크기 등을 선언할 수 있습니다. [33] SDKB v1.0 릴리스 번들은 “다형식 제공 + 검증 규칙 + 예제 질의”를 포함해야 재사용성이 높습니다.

파일/폴더	형식	레이어	설명
ontology/sdkb-core.ttl	Turtle	Open Core	핵심 클래스/관계/개념
ontology/sdkb-governance.ttl	Turtle	Open Core	BIS/NIST/ECHA 규칙 노드
ontology/sdkb-links-semi.ttl	Turtle	Link-Only	SEMI 표준 참조(식별자/링크만)
data/nodes.parquet	Parquet	Open Core	노드 테이블
data/edges.parquet	Parquet	Open Core	엣지 테이블
mappings/semikong_alignment.tsv	TSV	Open Core	SemiKong 매핑 결과
mappings/semicononto_alignment.ttl	TTL	Open Core	SemicONTO 매핑 결과

파일/폴더	형식	레이어	설명
validation/shapes.ttl	TTL	Open Core	SHACL shapes
provenance/prov.ttl	TTL	Open Core	PROV-O 활동/에이전트/출처
README.md	Markdown	Open Core	Dataset Card(윤리/편향/라이선스/limits)
LICENSE.txt	Text	Open Core	CDLA-Permissive-2.0 전문 또는 공식 링크
CHANGELOG.md	Markdown	Open Core	버전 변경이력
examples/sparql/	.rq	Open Core	예제 질의 모음

CDLA-Permissive-2.0 는 SPDX 에 정식 등록되어 식별자/전문을 제공하며, 재배포 시 라이선스 텍스트 제공(또는 링크) 등 실무적 요건이 있습니다. [10] 특히 CC-BY-4.0 소스를 함께 포함하는 경우, CDLA 호환성 가이드가 “적절한 Attribution 유지” 조건을 명시하므로, provenance_sources 에 **저작자/서지/라이선스 필드**를 강제하는 것이 안전합니다. [34]

활용 시나리오 및 평가 지표(질의 예 포함)

시나리오(대표 4 개)

- 1) **전문가 검색**: (한/영 동의어 포함) “EUV 레지스트” 검색 → 관련 SubProcess/Material/FailureMode 연결 탐색
- 2) **결함 진단**: “특정 공정에서 관측된 불량”의 원인/완화 경로 추출
- 3) **기술 포트폴리오/갭 분석**: 기술노드(<=7nm)에서 요구 스킬/계측/재료의 결손 탐지
- 4) **규제 리스크 질의**: 특정 장비 클래스를 입력하면 BIS 규칙(예: §744.23) 적용 가능성과 Entity List 연계 리스크를 반환

- Entity List 는 Part 744 Supplement No. 4 로 관리되며, 거래 당사자가 포함되면 추가 라이선스 요구 가능성이 있습니다. [28]

- \$744.23 은 반도체 제조장비(SME) 및 advanced-node ICs 관련 end-use 통제를 규정합니다. [29]

평가 지표(권장)

- 정합 품질: 매핑 Precision/Recall/F1(골드셋 기반), “경계 사례” 재검토율
- FMEA 추론 정확도: SME 판정 정확도, 원인 Top-k hit rate
- 품질/재현성: SHACL 통과율(=릴리스 기준), CI 재현 빌드 성공률
- 성능: SPARQL 질의 응답시간(p95), 그래프 로딩 시간
- 거버넌스 최신성: BIS/NIST/ECHA 규정 업데이트 반영 리드타임(주 단위)

SPARQL 예시 쿼리 3 개(요청 반영)

아래 쿼리는 SDKB v1.0 에서 권장하는 RDF 스키마(설계안)를 기준으로 작성한 예시입니다. 실제 IRI/속성명은 sdkb-core.owl 확정본에 맞춰 조정합니다.

규제 리스크 질의(BIS 규칙+ ECCN 연계)

```
PREFIX sdkb: <https://example.org/sdkb/>
PREFIX dcterms: <http://purl.org/dc/terms/>

SELECT ?equipClass ?eccn ?earRule ?ruleRef
WHERE {
  ?equipClass a sdkb:EquipmentClass ;
              sdkb:hasECCN ?eccn .
  ?eccn a sdkb:RegulatedItem ;
        sdkb:controlledBy ?earRule .
  ?earRule a sdkb:EARRule ;
           dcterms:bibliographicCitation ?ruleRef .
}
ORDER BY ?equipClass ?eccn
```

FMEA 경로 추출(불량→원인→완화+ 공정 단계)

```
PREFIX sdkb: <https://example.org/sdkb/>

SELECT ?subprocess ?failureMode ?rootCause ?mitigation ?conf1 ?conf2
WHERE {
  ?failureMode a sdkb:FailureMode ;
               sdkb:occursAtProcessStep ?subprocess ;
               sdkb:isDueTo ?rootCause ;
               sdkb:mitigatedBy ?mitigation .

  OPTIONAL { ?failureMode sdkb:confidence_isDueTo ?conf1 . }
  OPTIONAL { ?failureMode sdkb:confidence_mitigatedBy ?conf2 . }
```



```

}
ORDER BY ?subprocess ?failureMode

기술 갭 탐지(기술노드 조건에서 “필수 역량/계측 누락” 탐지)
PREFIX sdkb: <https://example.org/sdkb/>

SELECT ?subprocess (COUNT(?skill) AS ?skillCnt) (COUNT(?metro) AS ?metroCnt)
WHERE {
  ?subprocess a sdkb:SubProcess ;
              sdkb:relevantForTechNode sdkb:technode/7nm .

  OPTIONAL { ?subprocess sdkb:requiresSkill ?skill . }
  OPTIONAL { ?subprocess sdkb:measuredBy ?metro . }
}
GROUP BY ?subprocess
HAVING (COUNT(?skill) = 0 || COUNT(?metro) = 0)
ORDER BY ASC(?skillCnt) ASC(?metroCnt)

```

위험·제약 및 완화책

리스크/제약	설명	영향	완화책(구체)
표준 유료문서(특히 SEMI/JEDEC/IRDS)	유료 PDF/접근제한 문서는 본문 반입 시 저작권/라이선스 위반 소지	공개 데이터셋 무력화	Link-Only 레이어로 분리, 본문/표미포함, 식별자/개정본/URL 만 유지 [35]
규제 변경(BIS/NIST/ECHA)	규정은 잦은 개정/공지로 롤이 변함	최신성 저하	규정 노드에 effective_date, retrieved_date 필수화, 업데이트 파이프라인(월 1 회) 운영; BIS Entity List/744.23 를 구조화해 반영 [36]
민감 데이터 포함 가능성	장비 모델/벤더 정보.케이스 기술이 특정 기업 내부정보로 오인될 위험	법/보안 이슈	security_level(PUBLIC/RESTRICTED) 필드 강제, 사례(cases)에서 공개 가능 텍스트만 유지, 리뷰 게이트 추가

리스크/제약	설명	영향	완화책(구체)
인과관계(원인- 불량-완화) 오류	FMEA 인과는 도메인 의존성이 크고 잘못 연결 시 의사결정 오류	신뢰도 하락	FBSFM 기반 구조화 + evidence/confidence 필수화, validation_required 기반 SME 검토 우선 [4]
정합 오류(동명 이의, 계층 불일치)	온톨로지 매칭은 대규모에서 충돌/불일치가 빈번	질의/추 론 오류	lexical+embedding 후보 + 논리 기반 진단(LogMap) + 대규모 매칭(AML) 혼합 적용 [37]
공개 라이선스 혼합(예: CC-BY + CDLA)	혼합 라이선스에서 Attribution 누락 시 문제	배포 중단	provenance_sources 에 라이선스/저작자 필수, CDLA 호환성 가이드에 따른 Attribution 유지 [34]

위 계획을 따르면, 베이스라인(semiconductor_v0_3)의 장점(14 개 타입 기반, provenance 통계/검증 플러그 내장, FMEA 그래프 포함)을 그대로 살리면서도, 규제/표준/보안까지 포함한 **기술경영 의사결정용 공개 데이터셋(Open Core)**을 **재배포 리스크 최소화** 방식으로 SDKB v1.0 까지 확장할 수 있습니다.

[1] PROV-O: The PROV Ontology

https://www.w3.org/TR/prov-o/?utm_source=chatgpt.com

[2] [9] [26] [35] E01000 - SEMI E10 - Specification for Definition and Measurement of Eq

https://store-us.semi.org/products/e01000-semi-e10-specification-for-definition-and-measurement-of-equipment-reliability-availability-and-maintainability-ram-and-utilization?utm_source=chatgpt.com

[3] [28] [36] EAR | Bureau of Industry and Security

https://www.bis.gov/entity-list?utm_source=chatgpt.com

[4] An ontological framework for the integration of system design and FMEA | AI EDAM | Cambridge Core

https://www.cambridge.org/core/journals/ai-edam/article/an-ontological-framework-for-the-integration-of-system-design-and-fmea/64A2560455F3ED94E0DB2E1A6D6DDC33?utm_source=chatgpt.com

[5] [32] Cover page | shacl | W3C standards and drafts | W3C

https://www.w3.org/TR/shacl/all/?utm_source=chatgpt.com

[6] [33] Dataset Cards · Hugging Face

https://huggingface.co/docs/hub/en/datasets-cards?utm_source=chatgpt.com

[7] [18] The NIST Cybersecurity Framework (CSF) 2.0 | NIST

https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20?utm_source=chatgpt.com

[8] [20] IR 8546, Cybersecurity Framework Version 2.0 Semiconductor Manufacturing Profile | CSRC

https://csrc.nist.gov/pubs/ir/8546/ipd?utm_source=chatgpt.com

[10] Community Data License Agreement Permissive 2.0 | Software Package Data Exchange (SPDX)

https://spdx.org/licenses/CDLA-Permissive-2.0.html?utm_source=chatgpt.com

[11] [37] LogMap 2.0: towards logic-based, scalable and interactive ontology matching | Nature Precedings

https://www.nature.com/articles/npre.2011.6670.1?utm_source=chatgpt.com

[12] AgreementMakerLight - Daniel Faria, Emanuel Santos, Booma Sowkarthiga Balasubramani, Marta C Silva, Francisco M Couto, Catia Pesquita, 2025

https://journals.sagepub.com/doi/10.3233/SW-233304?utm_source=chatgpt.com

[13] GitHub - aitomatic/semikong: First Open-Source Industry-Specific Model for Semiconductors

https://github.com/aitomatic/semikong?utm_source=chatgpt.com

[14] [2411.13802] SemiKong: Curating, Training, and Evaluating A Semiconductor Industry-Specific Large Language Model

<https://arxiv.org/abs/2411.13802>

[15] A template for the arxiv style

<https://arxiv.org/pdf/2411.13802>

[16] CEUR-WS.org/Vol-3760 - Semantic Materials Science 2024

https://ceur-ws.org/Vol-3760/?utm_source=chatgpt.com

[17] Initial development of an ontology for the

https://ceur-ws.org/Vol-3760/paper12.pdf?utm_source=chatgpt.com

[19] NIST Releases Version 2.0 of Landmark Cybersecurity Framework | NIST

https://www.nist.gov/news-events/news/2024/02/nist-releases-version-20-landmark-cybersecurity-framework?utm_source=chatgpt.com

[21] SCIP - ECHA

https://www.echa.europa.eu/scip?utm_source=chatgpt.com

[22] [23] MatKG: An autonomously generated knowledge graph in Material Science | Scientific Data

https://www.nature.com/articles/s41597-024-03039-z?utm_source=chatgpt.com

[24] Wikidata:Licensing - Wikidata

https://www.wikidata.org/wiki/Wikidata%3ALicensing?utm_source=chatgpt.com

[25] GitHub - huanyu-li/SemicONTO: An ontology for the semiconductor domain · GitHub

<https://github.com/huanyu-li/SemicONTO>

[27] SECS/GEM Tutorial Seminar Overview | SEMI

https://www.semi.org/en/node/128996?utm_source=chatgpt.com

[29] 15 CFR § 744.23 - “Supercomputer,” “advanced-node integrated circuits,” and semiconductor manufacturing equipment end use controls. | Electronic Code of Federal Regulations (e-CFR) | US Law | LII / Legal Information Institute

https://www.law.cornell.edu/cfr/text/15/744.23?utm_source=chatgpt.com

[30] JSON-LD 1.1

https://www.w3.org/TR/json-ld11/?utm_source=chatgpt.com

[31] Paper page - Knowledge Graph Enhanced Retrieval-Augmented Generation for Failure Mode and Effects Analysis

https://huggingface.co/papers/2406.18114?utm_source=chatgpt.com

[34] Compatibility - CDLA

https://cdla.dev/faq-resources/compatibility/?utm_source=chatgpt.com